

Failure (bad key + cost comparison) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (turning a token count into a real budget decision). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The two parts, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The two parts, and the exact pieces

Basic Version

Two separate things to try here: 1. Use a fake API key on purpose and see exactly what error comes back. 2. Ask the model the same question twice — once with a short system prompt, once with a long one — and compare how many tokens each one used.

`OpenAI(api_key="fake-key-123")` — make a client with a wrong key on purpose. Wrap the call in `try/except`, catching the specific auth error, not a bare `except`. `response.usage` has the token counts on it after any real, successful call.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Part 1 tests that you catch `openai.AuthenticationError` *specifically*, not just any exception — because a bare `except Exception` would also hide a real bug in your own code behind the same generic message, making it much harder to tell "bad key" apart from "I broke something."

`OpenAI(api_key="sk-fake-key-123")` — passing a bad key directly to the client constructor (instead of relying on the real one from `.env`) forces an authentication failure on the very next call.

Part 2 uses `response.usage` — every response object carries back the exact token counts OpenAI billed you for, split into `prompt_tokens`, `completion_tokens`, and `total_tokens`. Comparing this number between a 1-sentence and a 20-sentence system prompt turns "shorter prompts cost less" from a rule you were told into a number you measured yourself.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

A token count is meaningful to you, right now, while you're reading it — but a dollar figure is what you'd actually put in front of a manager, or use to decide between two designs months from now.

`response.usage.prompt_tokens` and `response.usage.completion_tokens` are billed at different rates (OpenAI prices input and output tokens separately, and output is usually more expensive) — a single "total tokens" number hides that difference.

The harder, real design question underneath this exercise: **should your app just measure cost after the fact, or actively enforce a limit on it?** A single call's cost difference between a short and long system prompt looks tiny in isolation — but multiply either one by a million real requests, and a bloated system prompt becomes a real, ongoing expense that nobody notices until a bill arrives. A production system often tracks *cumulative* spend across a session (or a day), not just per-call cost, and refuses new calls once a budget is hit — the same "fail loudly, on purpose, before something bad happens" idea as Doc01's `require_env()`, applied to money instead of missing config.

```
PRICE_PER_1M_INPUT = 0.15
PRICE_PER_1M_OUTPUT = 0.60

def estimate_cost(usage) -> float:
    input_cost = (usage.prompt_tokens / 1_000_000) * PRICE_PER_1M_INPUT
    output_cost = (usage.completion_tokens / 1_000_000) * PRICE_PER_1M_OUTPUT
    return input_cost + output_cost
```

Sketch a small `BudgetTracker` yourself — something that adds up `estimate_cost(...)` across calls and refuses once a limit is hit — before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both measure a single call's token count and compare two numbers side by side. Advanced turns those token counts into an actual dollar estimate (since input and output tokens are priced differently, a raw token count alone can be misleading), and asks the bigger question a single comparison never raises on its own — whether your app should just *report* cost after each call, or actively *enforce* a spending limit across many calls, the way a real production system has to.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
part 1:
    make a client with a fake key
    try to send a message
    except the specific "bad key" error:
        print a clean message

part 2:
    ask the same question with a short system prompt, note the token count
    ask the same question with a long system prompt, note the token count
    compare the two numbers
```

Here's almost the whole thing for part 1:

```
import openai
from openai import OpenAI

bad_client = OpenAI(api_key="sk-fake-key-123")
try:
    bad_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": "hi"}],
    )
except openai.AuthenticationError:
    print("Login failed – check your API key.")
```

Expected output: Login failed – check your API key. Now write part 2 yourself: two calls, one short system prompt, one long, comparing `.usage.total_tokens`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
function try_bad_key():
    bad_client = OpenAI(api_key="sk-fake-key-123")
    try: send a message
    except openai.AuthenticationError: print a clean message

function compare_prompt_cost(question):
    short = call with a short system prompt
    long = call with a long system prompt
    print short.usage.total_tokens
    print long.usage.total_tokens
```

```
import openai
from openai import OpenAI

client = OpenAI()

def try_bad_key() -> None:
    bad_client = OpenAI(api_key="sk-fake-key-123")
    try:
        bad_client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": "hi"}],
        )
    except openai.AuthenticationError:
        print("Login failed – check your API key.")

def compare_prompt_cost(question: str) -> None:
    short_prompt = "Be helpful."
    long_prompt = "You are a friendly, detailed, thorough assistant who explains

    short = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": short_prompt}, {"role": "user", "c
    )
    long = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": long_prompt}, {"role": "user", "c
```

```
)
```

```
print(f"Short system prompt: {short.usage.total_tokens} tokens")  
print(f"Long system prompt: {long.usage.total_tokens} tokens")
```

Call both functions and confirm the long prompt uses noticeably more tokens before moving on.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
class BudgetTracker:
    __init__(limit_dollars): store limit, start spent at 0

    method record(usage) -> float:
        cost = estimate_cost(usage)
        if spent + cost > limit: raise BudgetExceededError
        spent += cost
        return cost

usage:
    tracker = BudgetTracker(limit_dollars=0.01)
    for each call you make:
        response = ...
        tracker.record(response.usage)    # raises once the limit is hit
```

Turning that into real code — fill in the missing piece yourself:

```
class BudgetExceededError(Exception):
    pass

PRICE_PER_1M_INPUT = 0.15
PRICE_PER_1M_OUTPUT = 0.60

def estimate_cost(usage) -> float:
    input_cost = (usage.prompt_tokens / 1_000_000) * PRICE_PER_1M_INPUT
    output_cost = (usage.completion_tokens / 1_000_000) * PRICE_PER_1M_OUTPUT
    return input_cost + output_cost

class BudgetTracker:
    def __init__(self, limit_dollars: float) -> None:
        self.limit_dollars = limit_dollars
        self.spent = 0.0

    def record(self, usage) -> float:
        cost = estimate_cost(usage)
        # your turn: if self.spent + cost would exceed self.limit_dollars,
```

```
# raise BudgetExceededError naming both numbers; otherwise add
# cost to self.spent and return it
...
```

Expected behavior: calling `record()` repeatedly with real `usage` objects adds up `self.spent` across calls, and the call that would push `self.spent` over `self.limit_dollars` raises `BudgetExceededError` instead of silently going over.

Fill in `record()` yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both measure and compare token counts for exactly 2 calls, side by side, then stop. Advanced turns those same `usage` objects into a running dollar total across as many calls as you make, and raises a clear, custom error the moment a caller would go over a set budget — the same "fail loudly, on purpose, before the bad thing happens" pattern Doc01's `require_env()` uses for missing config, applied here to spending instead.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)